Italian Development Guide

On iKernel OGAPI v2.20.0.0

Release Date: 14/02/2020

Document Version: 1.5

Status: Live

Introduction

This document is designed to guide new developers through use of the iKernel API and compliance with Italian legislative requirements by way of iKernel features. The document also details the use of the iKernel development kits on a standard PC.

Version History

Date	Version	Description	Author
07/04/2014	0.1	Initial draft	Leo Hancock
19/05/2014	1.0	First release	Leo Hancock
14/07/2015	1.1	Changes made after market spec update	Matt Flockhart
13/06/2017	1.2	Revising document	Rich Reynolds
29/06/2018	1.3	Updated to include information on Recovery. Game cycle no longer required to be completely predetermined APIFlush now required during exit Passing of GameCycleId into relevant calls now required Stipulates that Progressive Jackpots are no longer required. Accounts for SOGEI no longer being the only permitted certification entity.	Leo Hancock
07/01/2019	1.4	Rewritten for OGAPI (2.17.0.3) – required to accommodate the new Player Session decree Removed section on processing Progressive Jackpots as OGAPI does not support these calls.	Leo Hancock
14/02/2020	1.5	Updated to OGAPI 2.20.0.0 Amended Italian interface version to _v2 Added details on new game-managed tax popups Removed references to SetEscrow/EscrowWinnings (no longer used) Added clarity to Checkpoint calls	Leo Hancock

Contents

Introduction 2

Version History 3

1. Setting up and using your Development Kit 6

1.1 Unpacking the SDK 6

1.2 Installing prerequisites 6

1.3 Starting the iKernel 7

1.4 Stopping the iKernel 7

1.5 Adding Games 7

1.6 Launching games 9

1.7 The Swing Peripheral Mount 10

1.7.1 Volume Control 10

1.7.2 Buttons and Lights 11

1.7.3 Receipt and Ticket Dispenser 12

1.7.4 Note Reader / Coin Mech 12

1.7.5 Door Switch (Unsuspending a suspended unit) 13

1.8 The Content Development Jackpot Server and Random Number Generator 15

2. Italian and iKernel precepts 17

3. Loading, Connection and Initialization 20

4. Interacting with the iKernel 23

4.1 Buttons and Lights 23

4.2 Credit 24

4.3 Customer Specific Branding 24

4.4 Tax Disclaimer Message 25

4.5 Default Stake Property 25

5. Game Cycles 27

5.1 AskContinue 28

5.2 StartGameCycle 28

5.3 ReserveStake / ReserveAdditionalStake 28

5.4 GetRandomInt32 / GetRandomInts32 29

5.5 EscrowWinnings 30

5.6 CommitStake 31

5.7 RollbackStake 31

5.8 SetCheckpoint 31

5.9 AwardWinnings 32

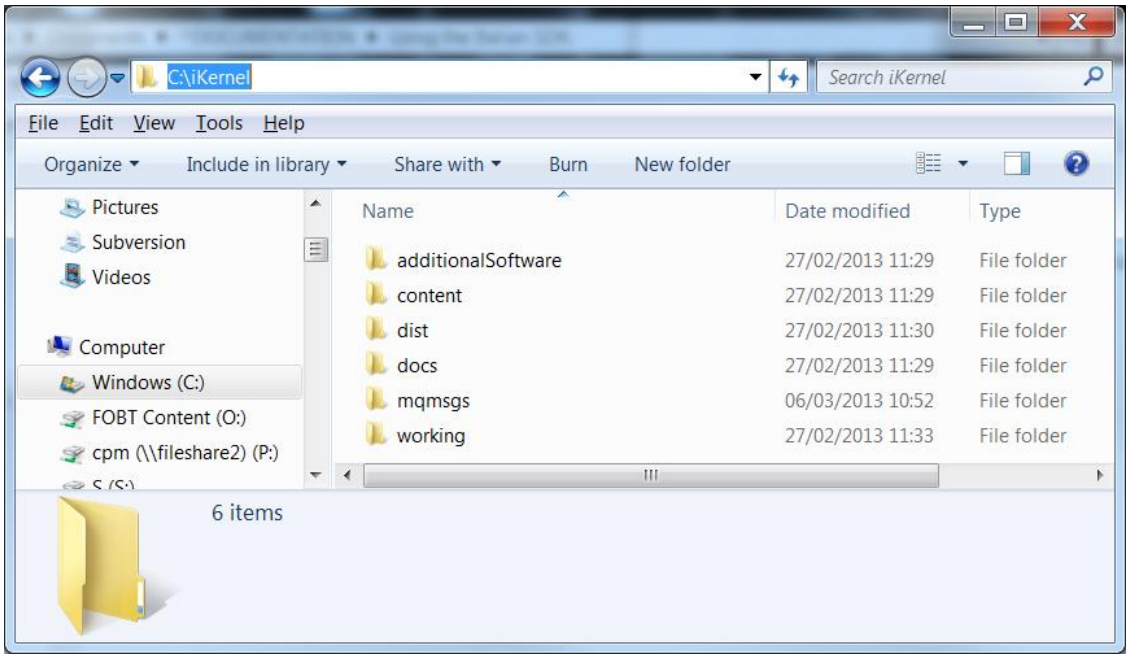
5.10 GameCycleEnd 33
5.11 Cashout 33
5.12 Extra Spins, Free Spins and other features 34
5.13 Quick Spin 34
5.14 OnExitDeferred and OnExit 34
6. Risk 36
7. Tax and Player Limits 37
8. Recovery 39
Terminated after StartGameCycle 40
Terminated after ReserveStake 40
Terminated after GetRandomInt32 40
Terminated after CommitStake 40
Terminated after AwardWinnings 40
9. Logging 42
10. Disconnecting and Shutdown 45

1. Setting up and using your Development Kit

The iKernel is a piece of software that runs a on Fixed Odds Betting Terminal or Video Lottery Terminal. It manages and abstracts the hardware, financial communication and keeps track of certain legal requirements games must adhere to. The software it's self runs on Microsoft Windows XP and Windows 7, but games must target Windows XP. The software may support Windows 10 but certain functions or features may be limited. There are no current plans to introduce support.

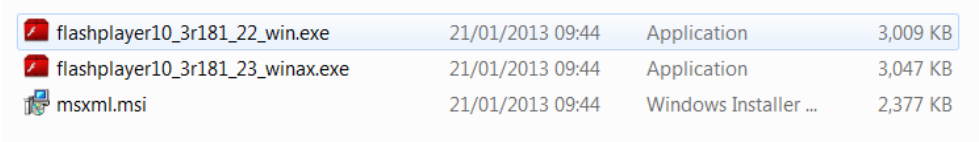
1.1 Unpacking the SDK

Your SDK will arrive in a ZIP file. Using your preferred archiving program, unzip the contents to the root of your C drive. WinRAR is not recommended as it cannot manage some of the long filenames included in the package. You should have a directory structure like the screenshot below (Docs and AdditionalSoftware can be supplied separately if they are not included in your package).



1.2 Installing prerequisites

Within the AdditionalSoftware directory, you will find 3 files. These must be installed before the iKernel SDK will function correctly. If you are using Windows Vista or Windows 7 and have User Account Control turned on, you may need to use an Administrator account to install.

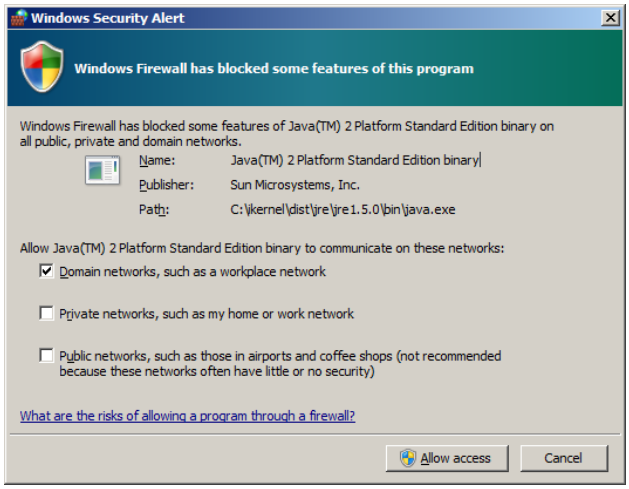


Two versions of Adobe Flash Player 10 are supplied. Each should be installed in turn. The final package included is Microsoft XML Parser 4.0 SP3.

1.3 Starting the iKernel

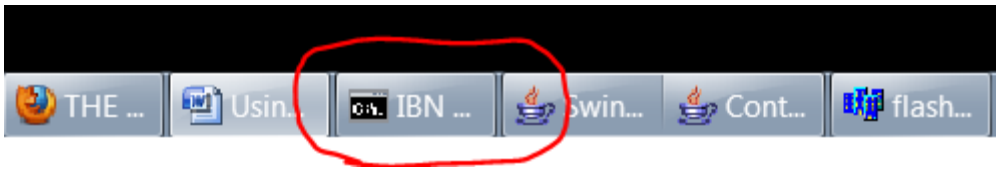
Navigate to C:\iKernel\dist\bin using either Windows Explorer or a command prompt. If you used Windows Explorer, locate and double-click upon iKernel.bat. If you used a command prompt, type "iKernel.bat" and press enter.

A command prompt window will appear and a large amount of text will scroll past. Note that you may receive an error message from Windows Firewall – you should allow access.



1.4 Stopping the iKernel

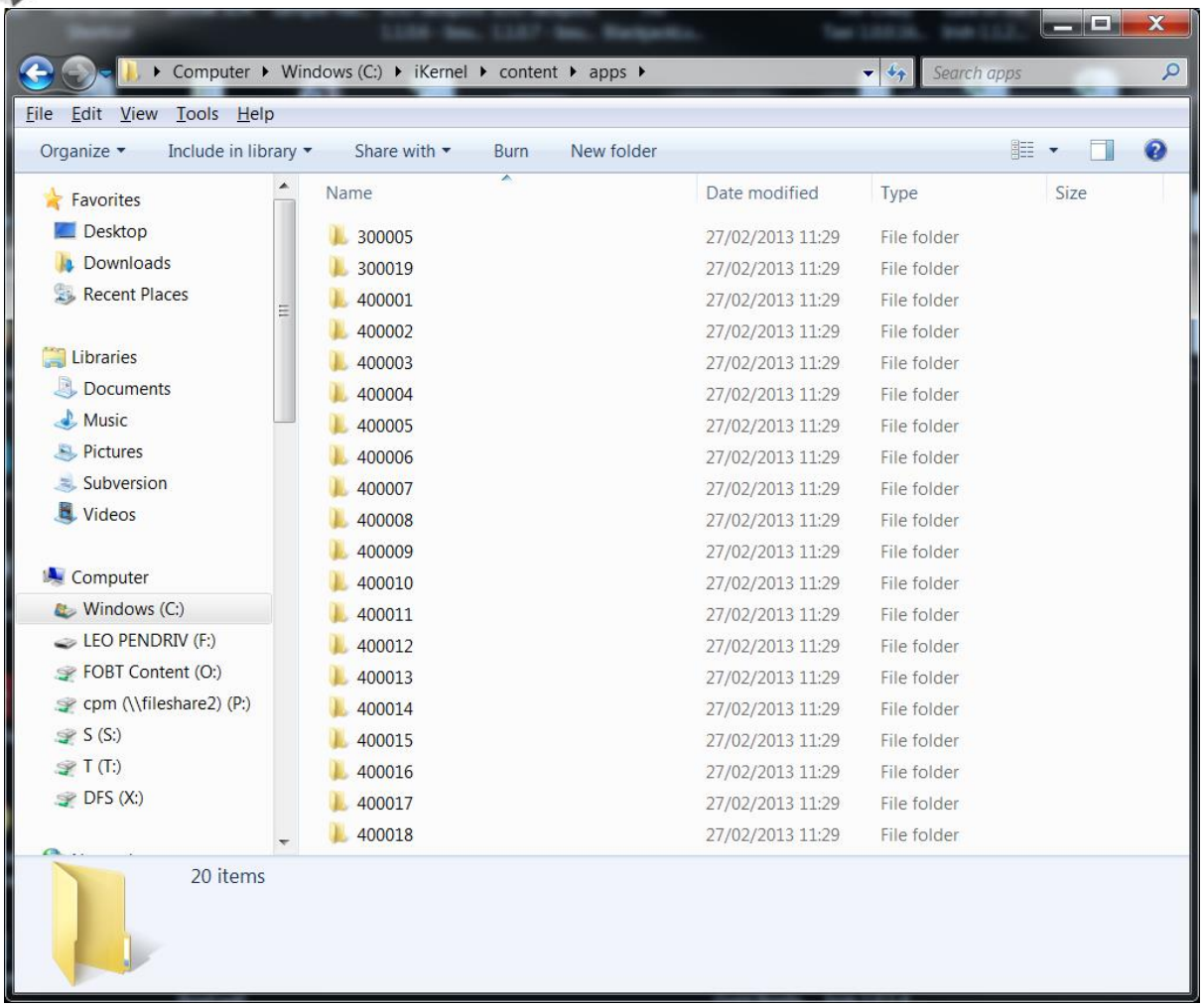
When you need to close the iKernel, select the command prompt window, hold the control key and press C. If you cannot select the command prompt window, press the Windows key and M to minimise all windows. You should then see the command prompt on the taskbar.



1.5 Adding Games

Note, before you add games it is recommended that you stop the iKernel if it is running. Games are placed within folders in the directory c:\iKernel\content\apps\. Several folders are present which contain components that the iKernel requires to function. Games are placed in the following folders depending upon what functionality is required:

400001 to 400012 have no access to Jackpots.
400013 to 400018 can access, contribute to and win Jackpots.



Note that newer versions of the development kit may show different numbers of available games.

Inside each numbered folder (this number is called the Content ID), you will find subdirectories 300019 and runtime. There is also a text file called content.properties which will not need modifying. The numbered folder contains the .swf file with the menu's button. Normally this is supplied with the game package built by INGG, but if developing your own it can be put into C:\iKernel\content\apps******\300019\it_IT\Widescreen\Sisal.swf.

The runtime folder contains the game itself along with all files it requires to run (configuration, graphics, DLLs etc). To install a new game from an INGG-supplied package, first delete both directories (don't delete content.properties) and then copy the two folders out of the package supplied. To install without a package, delete all the files inside the Runtime folder and replace them with the files for your game.

Your game executable must be named "game.exe" otherwise it will not function. If you have not named it correctly, the iKernel will not display its button on the menu. If it is named something different, you will need to rename it before you start the iKernel. Packages supplied by INGG will have the game named correctly.

1.6 Launching games

First, start the iKernel per instructions in point 1.3. You are presented with the following menu (note that if you have installed games from packages, the buttons may be different).

Games installed in folder 400001 to 400006 appear under Giochi di Carte, 400007 to 400012 under Giochi Slot and 400013 to 400018 under Giochi Slot Con Jackpot. A button without a game installed will launch Notepad or will fail to function. To return to the menu after launching notepad, simply close it and the menu will return. To return to the menu from a game, click the game's menu button.

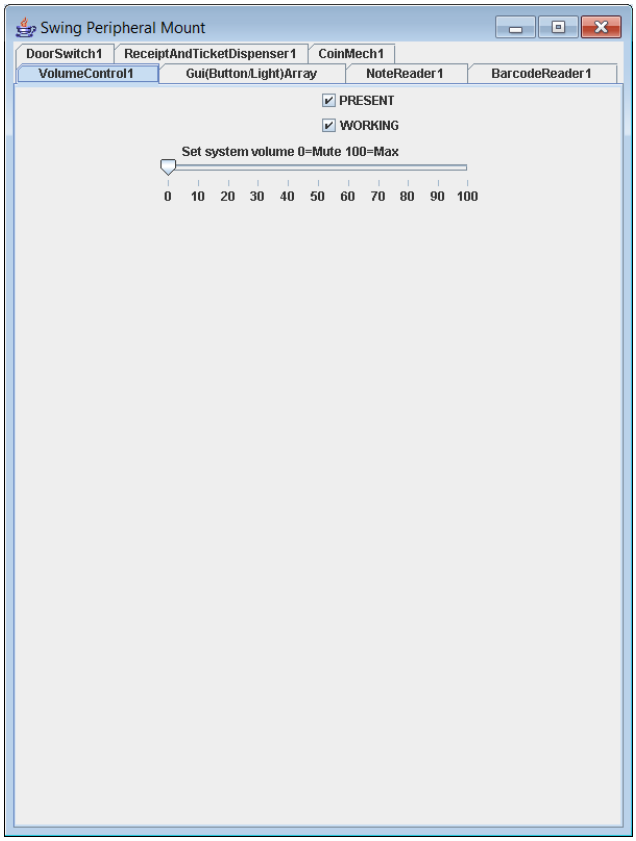
The menu layout may differ slightly in newer iterations of the development kit.

Games will not appear on the menu if they are not 'enabled'. By default all the games pre-installed are enabled. Deleting the working folder will result in all games being disabled – you can restore the "CLEANWorking" folder to return the development kit to its original state.

Please note that it is simpler by far to use the folders provided for your use. If you elect to install new games or are unable to restore the clean working folder, you can open the file located at c:\ikernel\working\302590\contentStateCache.txt . You can create this file if it doesn't exist. For each game that you need to appear on the menu, you need two lines:
400001.ENABLED=true
400001.INSTALLED=true
Replacing the number with the numbered folder of the game you have created.

1.7 The Swing Peripheral Mount

This application is intended to replace the functions of a physical cabinet, reacting to instructions from the game for things like lights and ticket printing, and sending instructions for things like button clicks and credit insertion. It will not be visible or will contain reduced functions if your development kit is installed onto a cabinet



The tabs along the top replace the named peripheral on the cabinet. Each has two check boxes, Present and Working. They are used to change the states of the peripherals. Unticking Present would be the equivalent of unplugging the peripheral from the cabinet, while unticking Working simulates a hardware failure. These options are unlikely to be required during game development or testing.

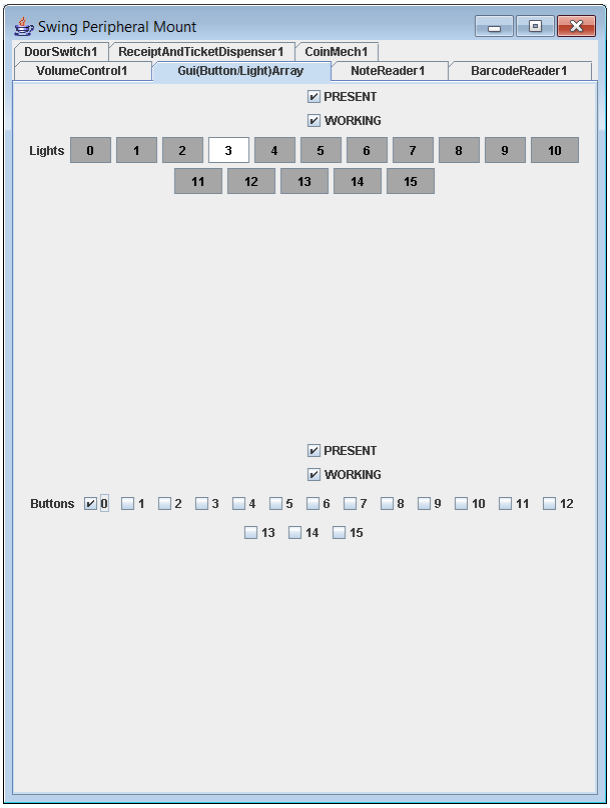
Note that the peripheral “BarcodeReader1” performs no function without other aspects of the iKernel that are not included in this SDK. It will not be shown in the instructions below.

1.7.1 Volume Control

The slider controls the volume of the Windows PC upon which the SDK is running. Its setting will not take hold until a game is launched. The volume level will not be restored upon leaving the game however. On some versions of Windows, this peripheral does not function.

1.7.2 Buttons and Lights

Lights show up white when lit, and grey when unlit. Buttons are represented by tick-boxes – when the box is ticked, the button is being held down. When unticked again, the button is released. To simulate a button press, the user needs to tick then untick the box.



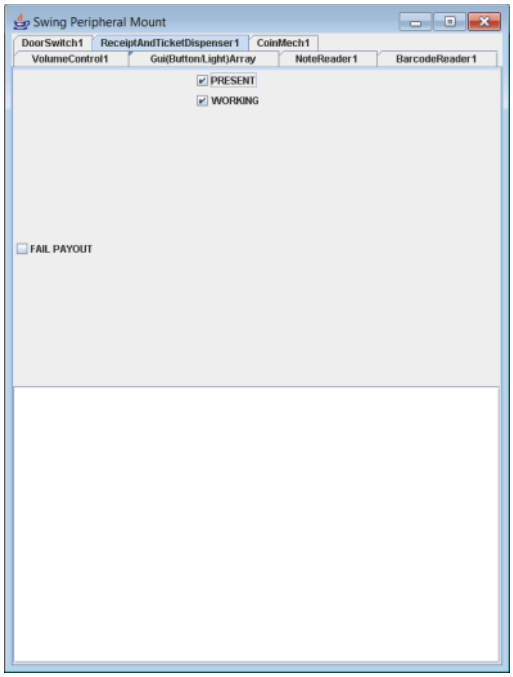
In this example, light 3 is lit and button 0 is held down.

The buttons and lights are numbered. Each number corresponds to a function, not all of which are applicable to the Italian market. The ones in use are as follows:

0 = Menu
2 = Start / Bet
3 = Collect / Ticket Out
7 = Max Bet
8 = Change Stake
9 = Gamble / Risk
11 = Stake Up
12 = Stake Down
13 = Lines Up
14 = Lines Down

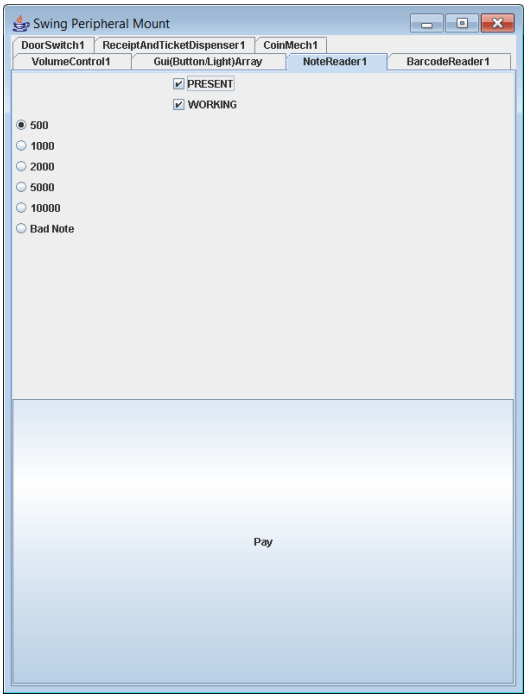
1.7.3 Receipt and Ticket Dispenser

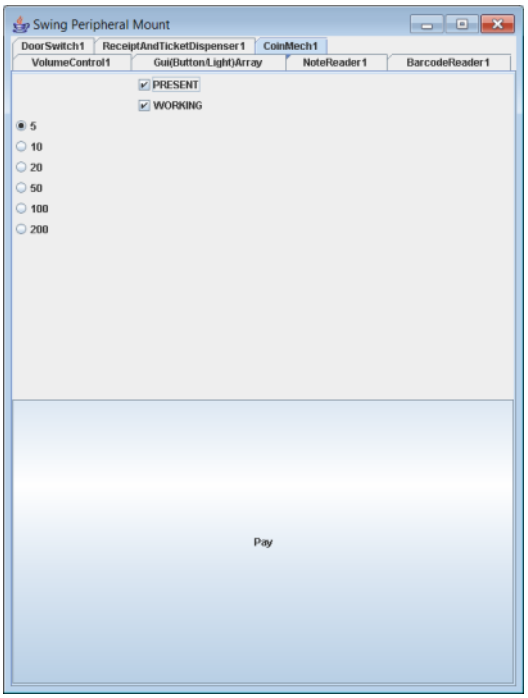
The only option here is to simulate a printer failure. The peripheral will report that the ticket has failed to print but will not give a reason code to the game. Italian regulations require that the cabinet is then suspended. If you do suspend the SDK, see below (Door Switch) for instructions on how to unsuspend.



1.7.4 Note Reader / Coin Mech

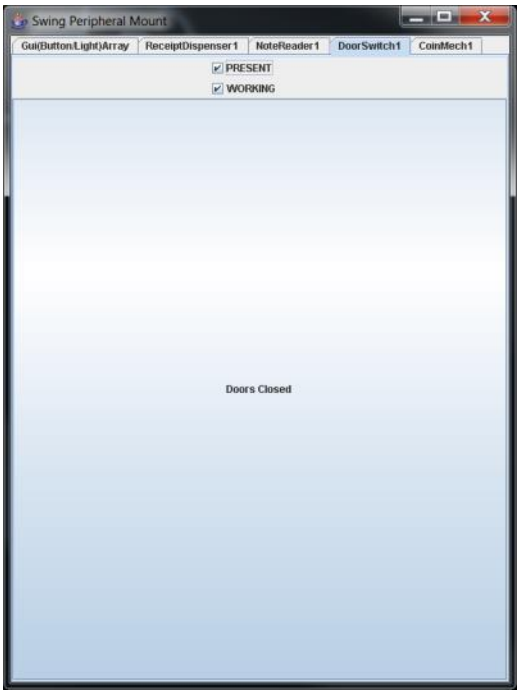
This allows the user to simulate the insertion of a note or coin into the cabinet. The radio buttons represent the available denominations of 5,10,20,50,100,200,500,1000,2000 and 5000 cents (€1, €2, €5, €10, €20 and €50). Having selected an option, the large “Pay” button at the bottom of the interface will pay the note into the SDK. Selecting “Bad Note” simulates the insertion of a note that the note acceptor rejects. It should have no bearing on the game.





1.7.5 Door Switch (Unsuspending a suspended unit)

The single large button simulates the opening and closing of the cabinet doors. Clicking it toggles the state from Doors Closed to Doors Open and vice versa.



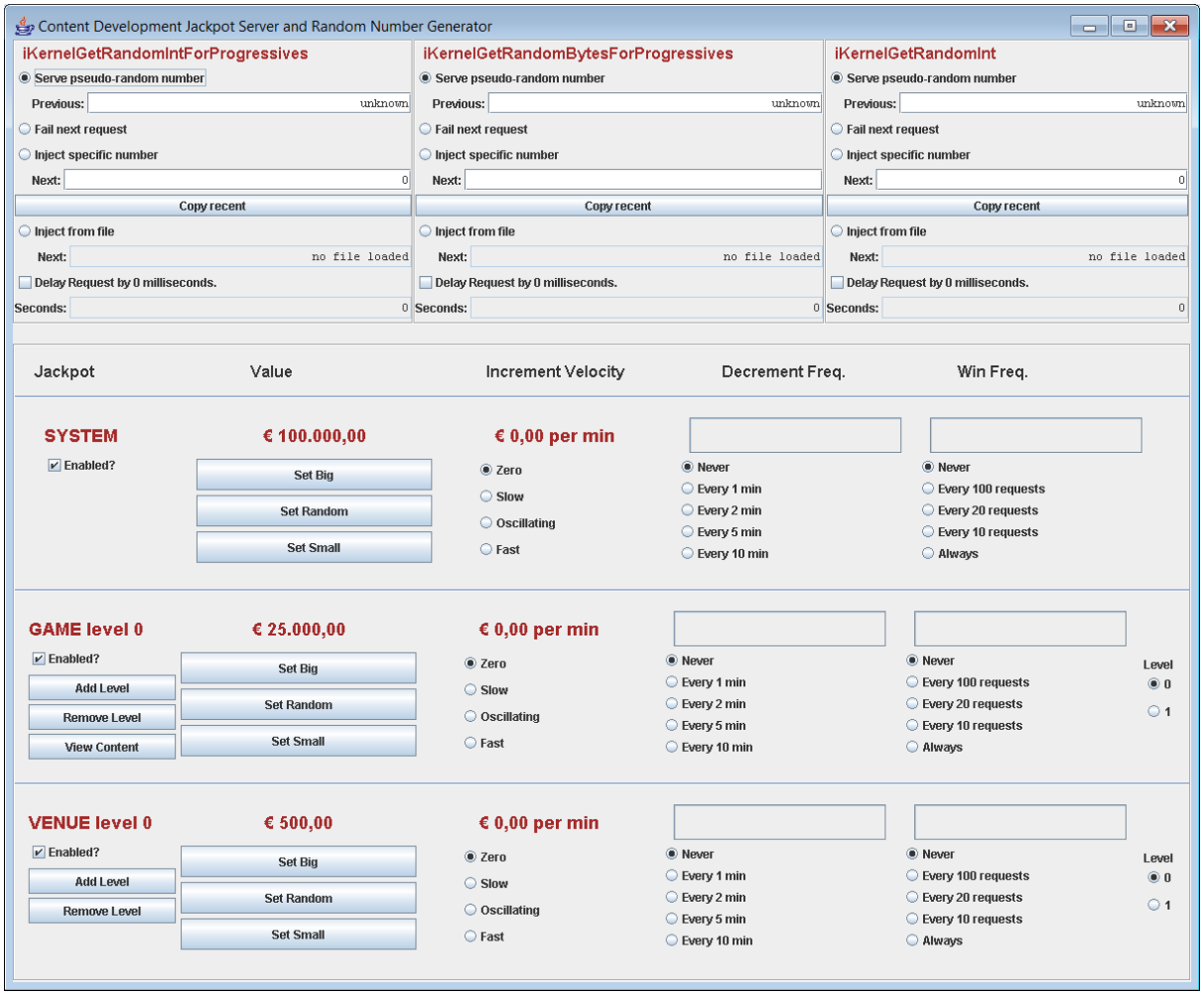
Once the doors are opened, the content menu closes and the engineer menu opens. This immediately puts the SDK into a suspended state and if you close the doors again without logging in, the SDK will not allow you to use it further. To unsuspend the unit, you must log in by clicking Log In and then entering the engineer username and password. For the SDK, these are set to 'Engineer' for

the username and ‘Engineer’ again for the password. Note that both username and password are case sensitive. The engineer menu will load, but this is not of use to a games developer or tester. If you are unable to log in with the above details, try ‘eng001’ for the username and ‘eng001’ for the password (again, both are case sensitive).

Once you have logged in and the engineer menu is being displayed, the terminal is unsuspended. You can continue to operate by clicking Doors Open to send the signal that the doors have been closed. The menu will reload after a short delay.

If you are unable to unsuspend the unit by this method, you can also delete the contents of the folder c:\iKernel\working\ but be aware that this will effectively factory-reset the unit, removing any saved information from games and your credit. You must then restore the contents from the “CLEANworking” folder supplied with your development kit.

1.8 The Content Development Jackpot Server and Random Number Generator



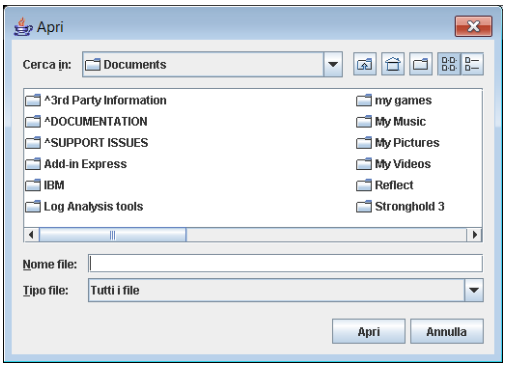
PLEASE NOTE that the Random Number Generator in the SDK still includes the ability to serve randomness for Progressive Jackpots, and varying levels of control for these jackpots. This document no longer describes their use is no longer required and is not supported by the updated API.

The top three boxes have the same options, selected by changing the radio button selections. Developers will only need to make use of the right-most box. The default will deliver a random number from the internal pseudo-random number generator when requested. The dialog box will update to show the number that was served.

Fail next request does exactly that; when the game requests a number the RNG will fail to operate, simulating a case such as a network outage.

Inject specific random number will cause the RNG to provide the exact number that is typed into the box instead of a random number. Pressing “Copy recent” will copy the value from “Previous” into the box.

Inject from File will let you specify your own sequence of numbers in a text file (enter the numbers with one number on each line). Clicking the radio button initiates the browse for file dialog box – use this to locate your text file.



You can cause the interface to delay the serving of a random number (in effect simulating network congestion) by checking the “Delay request by 0 milliseconds” box. You then type a value into the box underneath and the interface will wait for that many milliseconds before it delivers a random number to the game. Note that the timing of this feature is not 100% accurate, the delay is often somewhat longer than the specified value.

2. Italian and iKernel precepts

Italian regulations require that the platform and connectors are certified either by SOGEI, the Italian gambling regulatory body or by a 3rd party test house approved by them. Where this document references "SOGEI", the term is used to describe either entity.

Due to this certification requirement, only one version of the OGAPI DLL is valid. Furthermore, SOGEI will only permit the builds of the DLLs that were compiled in their software laboratories to be used in games destined for deployment in Italy. If you are in any doubt as to the correct version and build of the DLL you have been supplied, please contact your Inspired representative. The MD5 hash of the valid DLL is to be confirmed.

You must draw your random numbers from the iKernel. Under no circumstances whatsoever can games use an internal random number generator, a seeded pseudo-RNG or any other means of acquiring random data. A new random number (or set of numbers) must be drawn for every game or risk. It is not permitted to store numbers for any reason. See "Game Cycles" below for more information.

Any mathematical bias in the game must be in favour of the player. You must be able to explain and prove this to SOGEI by way of your documentation.

If a game is presented the same random number, it must award the same cash value. For the purposes of random numbers, Risk is considered a new game. See "Risk" below for more information.

Before your game is released, you will be required to present your source code to SOGEI. You are also required to extensively document your game (documentation requirements are covered in a separate guide). It is important to ensure that the code you write is of good, readable quality and fully understood by the coder you send to SOGEI, as they will be expected to be able to explain any nuance that SOGEI pick up on. You will also be required to explain how and why you make use of any 3rd party libraries or toolsets you use. For more information, refer to document "General Requirements and Documentation".

Your source code (including comments) must not make any mention of any other game, locale or territory and must not include any comments or references that imply that anything is unfinished (such as "To-do" or "hack"). If your code-base is written on a common library, you should pare this down to remove any such references and any unused code. Your code must be thoroughly commented with high quality comments.

While at SOGEI, once you pass the sourcecode review stage they will seek to compile your game. You will have provided a virtual machine which must contain the tool you use to compile your code and build the executable. As part of the package submitted to SOGEI, the runtime folder which will be deployed to terminals will be presented with the game's executable and the configuration file used to set the RTP removed. The executable that SOGEI compile will be placed into this folder and expected to run with no additional alterations. No other .exe file may be present either in your runtime, sourcecode or produced as part of your build process. If you use 3rd party libraries, you must ensure that any example executables they provide are removed from your sourcecode package and runtime folder.

Notwithstanding the above, it is permitted for other dependant files to be built as part of your build process (such as DLLs that are an intrinsic part of the game) provided they are also removed from

the runtime before it is presented to SOGEI. The runtime presented must contain only pre-built assets.

Games must target Microsoft Windows XP. The operating system will have DirectX 9, the XVid and Windows Media codecs available but games must provide any other extensions of this nature they require along with their runtime package. It is not possible to install anything to the terminals. The use of OpenGL is not supported.

Two screen sizes are in use, 1680*1050 and 1920*1080. In both machines, the screens are laid out thus:
[2]
[1]
where 1 is the main touch screen and 2 is the topscreen. Games must natively support both resolutions without requiring an update to the package. Although screens are currently arranged per the above, the game should not require this and should run regardless of where the screens are placed relative to each other. Under no circumstances may the game change the screen resolution.

It is important to remember that most of the interaction with the iKernel is asynchronous – you will call a method in the iKernel and immediately receive a success/failure message which indicates whether the call has succeeded, however you will not receive the actual response (by way of C-style callback) until much later.

Most of the calls you make to the iKernel must block the progress of the game cycle. Under normal circumstances this would present a poor player experience (for example, player presses Start, you request a random number then must wait for the response). To work around this, you are advised to continue your render loop while you await a response and to conceal the delay by doing things such as starting the reels of a slot game spinning as soon as Start is pressed. This is only one example of such an experience and a single solution to it but the experience to the player should be borne in mind and the tools in the development kit used to simulate as many worst-case scenarios as possible. This particularly applies to network latency, as the response for a random number can take several seconds at times.

At all times, it is not permitted to repeat the same call for no purpose. Games must not repeatedly call “SetLights” unless the light map has changed and must make use of the “OnCreditChangedEvent” callback rather than calling “GetCredit” every frame. For all that it makes simpler code, it is incorrect and the game will fail Inspired testing as it would not pass Integration post-SOGEI.

Games are presented with a ‘working’ folder by the platform. This is the only place they are permitted to write any files, logs, recovery data, non-volatile RAM or indeed anything at all. This directory is not adjacent to the game’s executable file. Games that write files to any other location including Windows temp folders) will fail testing. It cannot be used to ‘install’ game components.

Upon failure of any calls made to iKernel, or if an error is returned later via callback, the game should not make this known to the player. Instead the error should be written to the logs and the game should exit. If you are not able to write your own logs (for example, if the call that failed is part of initialization) iKernel will capture ‘stdout’ and ‘stderr’, but these must not be used once you can write to your own log file. If you are inside a game cycle at this time, crucially you must NOT end the game cycle as recovery relies upon the game cycle remaining open to decide whether or not to relaunch the game.

In addition to receiving calls as responses to methods you call, the iKernel can present the game with arbitrary events at any point via the callbacks the game registers. Games must be ready to receive and if necessary react to an event at any point. The events themselves are covered in more detail elsewhere in this document. Callbacks are only triggered during calls to ProcessCallbacks. Bear in mind therefore that this call will appear at the top of the call stack if you are debugging, but it is extremely unlikely to be this actual call which is causing the issue, it will be one of the callbacks that is triggered by the call.

Several callbacks share the same prototype. You can elect either to provide a separate callback for each method using the same prototype, or you can use the userData field provided in the call to identify the originating function, and switch the result based on that field.

Every call that takes a callback as an argument also has a userData field. This argument can be null but cannot be omitted. Each callback follows the same basic structure:

```
callback(void *userData, const ErrorInfo *errorData, 0 or more other arguments)
```

userData is passed back to you completely unmodified – you should get back exactly what you pass when you make the call that results in the callback. errorData will be null if there are no issues with the callback. If it is not null, the contents of errorData will provide more details. It is useful for debugging purposes to log both fields. In almost all cases the receipt of non-Null errorData should indicate a fatal exit to the game – any exceptions to this are detailed below.

The game must maintain no timeout on responses from the iKernel as this is handled by the DLL. The game will always eventually receive a response even if that response is a failure.

3. Loading, Connection and Initialization

The game connects to the iKernel by way of the OGAPI (Open Gaming API) DLL named OgapiDLL_Release.dll. Various header files are included alongside the DLL which should be included in the normal manner. **All interaction with the OGAPI DLL must be performed on your main thread – it is not thread-safe.**

The RequestAdminInterface function is the only function available on the DLL. It is used to retrieve the Admin interface and that interface can then be used to access all the other available OGAPI interfaces.

The API is presented as a collection C interfaces. Each interface comprises of a struct containing function pointers.

When content wishes to interact with platform it must first request the appropriate interface from the DLL and use that to perform the actions required. These calls are almost exclusively non-blocking calls that provide a callback for receiving the response. Callbacks (including those that you register to receive arbitrary events) are only ever triggered during a call to ProcessCallbacks.

To load the DLL, the following code should be used.

```
void* dll_handle = ::LoadLibraryA( "ogapiDLL_Release.dll" );
if ( !dll_handle )
{
exit( 1 ); // Failed to load the dll correctly.
}
const ::HMODULE module = reinterpret_cast< ::HMODULE >( dll_handle );
const ::FARPROC proc = ::GetProcAddress( module, "RequestAdminInterface" );
InterfaceRequesterType RequestAdminInterface = reinterpret_cast<
InterfaceRequesterType >( proc );
const OGAPI_Admin_Interface_v2* adminInterface = (
OGAPI_Admin_Interface_v2* ) RequestAdminInterface( "admin_v2" );
```

Once the Admin interface has been requested, it is used to request subsequent interfaces. For Italy, you require:

```
OGAPI_CabinetButtons_Interface_v1* CabinetButtonsInterface =
(OGAPI_CabinetButtons_Interface_v1*) adminInterface->RequestInterface(
"cabinet_buttons_v1" );
```

And

```
OGAPI_LegionItComma6_Interface_v2* LegionItalyInterface =
(OGAPI_LegionItComma6_Interface_v2*) adminInterface->RequestInterface(
"legion_it_comma_6_v2" );
```

You can name the instance of your interface however you choose, the above are suggestions only. Once you have requested all three interfaces, Admin and CabinetButtons interfaces must be initialised.

```
bool adminInterface-> Initialize();
bool CabinetButtonsInterface-> Initialize()
```

Bear in mind you cannot request additional interfaces once you have initialised the admin interface so you will need to request all interfaces before initialising any. Initialise returns a bool and takes no parameters. If the return is false there has been a critical error and the game should exit immediately. The `LegionItalyInterface` does not require initialising.

Beginning immediately after you call initialise on the admin interface, you need to call
```
bool adminInterface-> processCallbacks ();
```
approximately once per frame. If you have a high framerate you may elect to call this less often but the responsiveness of your game may suffer. Note that the CabinetButtons interface has its own processCallbacks method which you will need to call also in order to receive physical button events. Again, if the return values are false on either of these calls it indicates a serious error (such as the DLL has lost contact with the platform) and the game should exit, logging if applicable.

It is extremely important to note that data received via the callback mechanism is only valid for the lifetime of that callback. Once the callback returns, data will be out of scope. Therefore if you need to make use of data outside of the callback you will need to deep copy it to a local variable – pointers will no longer be valid. You should also note that while debugging, ProcessCallbacks will be at the top of the call stack – it is almost impossible however for this function it's self to cause the error – it is most likely that one of the callbacks it has invoked within your own code is causing the fault.

It is best practice to begin to display your loading graphic as soon as you can. The display of a prolonged black screen to the player presents a very poor player experience. Games should take no longer than 5 seconds from the moment they are selected from the menu to the time the player can begin a game cycle. Delayed loading of assets is permitted if there is no detrimental effect to the player. Subsequent loads must take no longer than 3 seconds.

From the moment `adminInterface->Initialize` is successful the iKernel will start to track your game's lifecycle. If it believes the game has hung it will terminate the process. To inform the iKernel you are alive, you need to send
```
void LegionItalyInterface -> heartbeat(CallbackWithNoData cb, void
*userData);
```
approximately every 15 seconds. You must continue to call this even while you are blocking execution waiting for a response from the iKernel or another callback. iKernel will begin the process of terminating the game if it doesn't hear a heartbeat within 25 seconds (30 in comes circumstances, but 25 is safer to assume).

You will then need to establish the working directory for your game as you will wish to start logging as soon as you can. You call
```
void LegionItalyInterface -> GetWriteableDirectory(CallbackWithStringInfo
cb, void *userData)
```

your callback will contain `const StringInfo* responseData` as its third parameter. This provides you the path to the writeable directory.

If you need to log before you are returned your writable directory, the iKernel will capture 'stdout' and 'stderr' and write to its own log file. This feature can cause issues with the platform if it is used to excess particularly with extremely long strings – minimal logging only should be sent to std and it should cease the moment you are able to log to your own working folder, with the sole exception of the need to log the inability to write to your log file.

You have complete control over this directory and may read, write and delete files and directories as you require. You may use this directory as a temp folder but must not exceed 200mb or a total of 200 files. You may not install files to this location (such as extracting game assets on first load). The working directory is empty on first boot and may be emptied at any point between game executions. The game must take adequate precautions to ensure that it can cope with this. Your recovery data must also be written to this folder.

Previous user of Inspired APIs will note that it is no longer necessary to subscribe to arbitrary events. This is all handled by the API DLL and events are sent to you by way of the callbacks you must declare that handle them. You must provide callbacks for

```
void LegionItalyInterface -> OnCashoutEndEvent (CallbackWithNoData cb, void
*userData)

void LegionItalyInterface -> OnCreditChangedEvent(CallbackWithNoData cb,
void *userData)

void LegionItalyInterface -> OnExitDeferredEvent(CallbackWithNoData cb,
void *userData)

void LegionItalyInterface -> OnExitEvent(CallbackWithNoData cb, void
*userData)

void LegionItalyInterface -> OnLimitsEvent(CallbackWithLimitsInfo cb, void
*userData)
```

These callbacks and the events to which they pertain are discussed in more detail below.

You will be asked to provide several different RTP variants for your game. The RTPs you must offer are detailed in the Market Specification and / or the Work Order. These must be configured by way of a setting within a single file which Inspired test can place in the game package. You will need to supply files preconfigured to each variant and your game will need to load and validate these at some point during the game's loading process. If the file is invalid the game must fail to load.

Next the game needs to call

```
LegionItalyInterface -> GetProperties (CallbackWithNamedValueArray cb, void
*userData)
```

The callback will be passed userData and errorData as ever, and a pointer to a NamedValueInfo struct. This contains array of NamedValue and an int indicating its length. NamedValue contains two char* , one for name and one for value. These are Content Properties, miscellaneous pieces of data passed from the platform to the game. The name and value should be stored to be acted upon later – the uses are described below in this document.

Once the game has finished loading, dropped its loading screen and is ready to accept player input, it must make a call to

```
LegionItalyInterface -> LoadingComplete (CallbackWithNoData cb, void
*userData)
```

OGAPI will not permit you to make gamecycle related calls until you have made this call, it will fail with invalid state errors.

4. Interacting with the iKernel

Your interactions with the iKernel are asynchronous. The API DLL wraps the detail of communications with the actual iKernel and presents you with the methods available in the header files. Responses are received at an unspecified future time by the callback mechanism while your main loop is calling `bool adminInterface-> processCallbacks ();` . You will not receive any communication from iKernel outside of this call.

Most communication with the iKernel can be said to be of 3 types. You will either be informing the iKernel of an event and you do not expect data in the response (such as starting a game cycle), polling the iKernel for current information (such as the Get Writable Directory call mentioned in initialization above) or receiving an event from the iKernel that you were not necessarily expecting. To see the latter, developers need to register a callback handler during their startup stages with the 5 "OnXxxxEvent" style of callbacks indicated above.

Developers will either wish to make use of the userData argument to pass a marker to a handler for each type of callback, or register a separate callback for each type. There are many different "CallbackWithNoData" and "CallbackWithStringData" methods that exist, and the manner of dealing with these should be established before you implement your solution.

4.1 Buttons and Lights

In order to receive button press information, the game needs to register a callback to be invoked by the iKernel when the game calls

```
CabinetButtonsInterface -> processCallbacks() ;
```

This should be called before or after adminInterface -> processCallacks() ; each frame, not during. The callback is registered by calling

```
CabinetButtonsInterface -> SetButtonCallback )(ButtonCallback cb, void
*userData)
```

Your ButtonCallback takes the inevitable userData, and two int values. These are used to indicate the button index the callback is referring to and the state of the button (0 is not currently depressed, 1 is currently depressed). ButtonCallback is invoked once per button that has changed state since the last call made to `CabinetButtonsInterface -> processCallbacks()`

In this manner it is possible (albeit unlikely) that you might receive a callback for a button that was in state 0 (not depressed) and is now also in state 0 (not depressed). You can assume in this case that the player has pressed and released it since your last ProcessCallbacks and take appropriate action

Button IDs and functions are as follows. The intended behaviour of each function is covered in the market specification document available separately:

ID Number	Function
0	Menu
1	Repeat Bet (Unused in Italy)
2	Start
3	Collect
4	Help (Unused in Italy)

5	High gamble (Unused in Italy)
6	Low gamble (Unused in Italy)
7	Max Bet
8	Change Stake
9	Risk (Gamble)
10	Auto Play (Unused in Italy)
11	Stake Up
12	Stake Down
13	Line Up
14	Line Down

Not all cabinets support all the buttons but games must to support the full range of machines to which they may be deployed.

The lights for each button correspond to the sane numbers as the above. A button must not be lit if it performs no function and must be lit if it has a function at that moment in time. There is no built-in function to flash a button. Any flashing patterns that you wish to apply to lights must be built by the game and realised by sending through different lightmaps at the correct interval for the pattern you wish to display.

Keep in mind that the iKernel will treat lighting of button lamps as the lowest priority for processing. As such, a fast flashing pattern will be subject to delay if the iKernel is busy, which will not look good. For this reason, Inspired do not recommend flashing button lights.

Lights are lit by calling

```
CabinetButtonsInterface -> SetLights (unsigned int lights)
```

The unsigned int is a 32 bit mask, where the least significant bit maps to 0 and the most significant to 31.

4.2 Credit

Initial credit upon load is retrieved by calling `LegionItalyInterface -> GetCredit(CallbackWithCreditInfo cb, void *userdata)` . The callback is invoked during ProcessCallbacks. Its 3rd parameter is a pointer to a CreditInfo object. This struct contains 3 int variables. The only one of interest to an Italian game is `stakeableCredit` – it is this value that the game should take and display in the credit meter. It should also be used to ensure that the correct amount of money is available before attempting to take a stake. The value is shown in the currency's minor unit (i.e. cents – a value of 150000 would be €1.500,00).

The value can change at any time. When it does, iKernel will invoke the callback the game has registered to `onCreditChangedEvent`. This callback is a `CallbackWithNoData` – as such it should be taken as an instruction to call `GetCredit` again, rather than the callback itself supplying you with any information. Credit should always be updated immediately.

4.3 Customer Specific Branding

Games should read the "DefaultCustomer" content property on load up. If the value is 'Sisal' then the Sisal-branded graphics should be used in place of the default:

```
If
```

```
DefaultCustomer value is supplied by the iKernel
then
{ If
DefaultCustomer == 'sisal'
then -> load the sisal branded version
else -> load generic unbranded version
}
else
{
load generic unbranded version
}
```

The game should be able to accept upper or lower case statements without issue.
Any other values read in through "DefaultCustomer" including no value or if the property is absent, should load in the generic graphic for the set limit button 'Imposta Limiti' graphic can be found in \Generic Game Graphics Pack\Set Limit Button\Generic. The 'Gioca Il Gusto' loading screen and help page should also not be shown.

During source review at SOGEI, we should be able to clearly demonstrate that any other string apart from 'SISAL' loads an unbranded version (with a different set of graphics files only), it should be obvious that if iKernel supplies any value other than 'SISAL' or no value/wrong value for DefaultCustomer, then the game must load but with a set of generic graphics i.e. game splash/loading screen.

4.4 Tax Disclaimer Message

A disclaimer message is required to be displayed on the bottom of the top screen in-game. To achieve this and make the message externally configurable (i.e., outside of SOGEI certification), the game should seek the content property 'MenuDisclaimer'. The game should look for the above-mentioned content property at load time. The value supplied by 'MenuDisclaimer' will be the text which is to be displayed on the top screen of games. Please use Arial (or similar), font size 9 to display the text. This text must centre aligned and aesthetically pleasing. Please note if this content property is not supplied or supplies a blank, then no message should be displayed and it should be obvious that game is expecting something to be displayed. Inspired games achieve this by displaying a box with a solid background which normally holds the message.

See "Tax and Player Limits" for more information on tax

4.5 Default Stake Property

At load time the game will look for content property value 'DefaultStake' and load the game as following…

```
If
DefaultStake value is supplied by the iKernel
then
{ If
DefaultStake == 'one of the allowed values' [in cents]
then -> load game with that default stake
else -> load game with lowest available stake
}
```

```
else
{
load game with lowest available stake
}
```

If an incorrect value or no value is provided via DefaultStake, then the game should launch with the 50c or lowest stake value.

5. Game Cycles

In the broadest possible terms, a game cycle is the moment from when the player presses Start to commit a stake to play, until the game returns to a state where it is ready for the player to press Start again (or to begin the next play if an Auto Play facility is in use) and all monies have been awarded or lost. This can incorporate features, Risk and / or in-game staking options (such as a Split in Blackjack). There are no enforced maximum times spent within or quantity of features / Free Spins / Extra Spins

A basic game cycle without a Risk runs thus. Note that a game cycle that does not generate a win will not call `AwardWinnings`. A game that has features and risk will make more than one call to `SetCheckpoint`. The calls are detailed below:

```
LegionItalyInterface -> AskContinue ( CallbackWithIntInfo cb, void
*userData )

LegionItalyInterface -> StartGameCycle ( CallbackWithStringInfo cb, void
*userData, const char *gameCycleId )

LegionItalyInterface -> ReserveStake (CallbackWithReservedStakeInfo cb,
void *userData, int amount)

LegionItalyInterface -> GetRandomInt32 ( CallbackWithIntInfo cb, void
*userData, int lowerBoundary, int upperBoundary )

LegionItalyInterface -> SetCheckpoint (CallbackWithNoData cb, void
*userData, const char *phaseSummary, const PhaseInfo phaseInfo)

LegionItalyInterface -> CommitStake ( CallbackWithNoData cb, void
*userData, const char *token )

LegionItalyInterface -> AwardWinnings ( CallbackWithNoData cb, void
*userData, int amount )

LegionItalyInterface -> EndGameCycle ( CallbackWithNoData cb, void
*userData, const char *gameCycleSummary, const PhaseInfo *phaseInfo )

LegionItalyInterface -> AskContinue ( CallbackWithIntInfo cb, void
*userData )


..

..
LegionItalyInterface -> AskContinue ( CallbackWithIntInfo cb, void
*userData )

LegionItalyInterface -> StartGameCycle ( CallbackWithStringInfo cb, void
*userData, const char *gameCycleId )
```

etc

For calls which require a response, the game must block game flow while waiting for the response to these calls. It must continue to render frames but must not make further calls to iKernel until the iKernel invokes your callback. The exception to this of course is `ProcessCallbacks`, which the game must continue to call, and the processing of events (delivered by the callbacks registered above by the "On…." Calls)

Since the introduction of the Recovery system, further calls to `GetRandomInt32` are permitted throughout the game cycle in order to determine the course of play. For games that require a larger amount of randomness, `GetRandomInts32` can be used.

5.1 AskContinue

Called before starting and after ending a game cycle to ensure the iKernel doesn't need the game to terminate for any reason. The callback contains an int to be treated as a bool – 1 for true, 0 for false. If the call fails or the callback contains either non-Null errorData or an int value of 0, the game must gracefully exit immediately, logging the reason. No message should be shown to the player.

5.2 StartGameCycle

Instructs the iKernel of the game's intention to start a game cycle. No credit handling may be performed before a game cycle is started, or after it is ended.

The call takes as a parameter a game cycle ID as a char* . If the game is recovering it should pass the ID of the cycle it is recovering in this parameter, otherwise pass null.

The callback contains a char* with the game cycle ID, either as passed to the call or newly generated. If the callback contains non-Null errorData or the call fails, the game must immediately exit gracefully and log the reason. No message is to be shown to the player.

5.3 ReserveStake / ReserveAdditionalStake

Reserves credit to the value passed in the method it so that may be later rolled back or committed. The credit change is automatically rolled back in the event of a power loss or game crash. Causes a Credit Change event to be fired to the game as the Playable credit is immediately reduced. The game will need to update the credit meter to reflect the reduction in credit (see "Credit" above). The legal minimum total stake (including any in-game betting) is 50c, and the maximum is €10.

ReserveStake can only be called once per game cycle – attempts to call it again will (intentionally) cause the OGAPI DLLs internal state machine to become invalidated. ReserveAdditionalStake can be called as many times as necessary up to the legal maximum. NOTE that the DLL does not enforce the legal maximum – the game bears this responsibility. Attempting to reserve more than the legal maximum will cause the call to fail and the terminal may be suspended.

The call should not be used to reserve stakes prior to starting a game cycle for a game where a variable stake is decided upon prior to starting a game cycle (such as placing chips on a Roulette table). The game will need to handle the display of credit available to stake vs actual credit on the machine internally in this instance.

The call takes the amount to reserve in minor units as a parameter. The callback will contain a ReservedStakeInfo struct. This contains a Status value (either OK or not authorised) and a token as

char* . The game should log the failure and exit if it receives unauthorised in the status (which will cause the iKernel to restart the game in recovery). It will need to store the token for use in later CommitStake.

It is particularly important that the game is aware of which token belongs to which staking action in a game that uses in-game betting so that the correct staking action is committed.

If the call fails or the callback contains non-Null errorData, the game must immediately log the fact and terminate gracefully. No message should be shown to the player. Upon success of the call it is normally appropriate to begin to display animations to the player (such as starting the reels spinning) as the rest of the calls required can take some seconds to return.
See also Tax and Player Limits and Recovery sections below.

5.4 GetRandomInt32 / GetRandomInts32

Requests a random number from the iKernel in the bounds passed in the call, with the results inclusive of the specified bounds. iKernel provides randomness using a 2s complement 32 bit signed integer, so everything from -2^31 to 2^31-1 giving 2^32 possible values if a range of LOWER=-2^31, UPPER=2^31-1 . The callback will contain the randomness requested as well as the usual errorData and userData fields.

GetRandomsInts32 should be used if the game requires multiple random numbers with the same upper and lower bounds to be able to determine the game cycle outcome. The call restricts you to 32 random numbers but the maximum amount of randomness that can be provided is 2^1024 . This means a single request with an upper bound of 2^1024, 2 calls up to 2^512, 4 of 2^256 and so forth can be requested with this call.

The game can make calls as required to either of these methods until it has sufficient randomness to start the game cycle. As the Italian iKernel requests randomness from a central location (with the game locally analysing the outcome in order to display the win) games need to be aware that the more randomness requested, the longer the callback(s) will take to get to the game, due to network latency. The random numbers that are received must never be logged to any file or displayed on screen.

How the game makes use of the random number is the province of your mathematicians. If you can satisfactorily explain your method to SOGEI and there is no bias (or if there is, the bias is as small as possible and is in favour of the player) you are not limited to any particular method. Inserting the same random number or sequence of random numbers into the game should result in payment of the same winnings regardless of how the game reaches that total i.e. the animation displayed to reach the win is permitted to vary. Since the advent of the Recovery system, the use of variable animations is no longer recommended for newly developed games.

It should not be assumed that the game will not receive the same number in any number of successive calls. Although the chances may be infinitesimal, Inspired Gaming's random number server is truly random – it is possible. Obviously with a smaller random bound this becomes more likely – a red / back card risk stage for example may choose to draw a number between 0 and 1.

If any of the calls fail or returns false, the game must immediately log the fact and display the message “Il terminale ha fallito la connessione al server RNG. La partita verrá terminata” on screen for 5 seconds before the game exits gracefully. Do not end the game cycle. iKernel will manage the return of credit in the event of a catastrophic failure of the game’s recovery system but as the game cycle is not ended, iKernel will restart the game which will be expected to attempt recovery. If there is a network outage, this is likely to occur until the terminal becomes suspended

5.5 CommitStake

Until this call succeeds, the credit deducted by the game can be rolled back by the iKernel in the instance of recovery or terminal failure. Games must not commit the staking action until they have escrowed the winnings from the game cycle based upon the randomness they have requested. This provides for a miniscule edge case where in the event of a power loss the player may be awarded the winnings from the game cycle as well as having their stake returned but as this is in favour of the player it is legally acceptable.

It must be called once for every staking action the game has called ReserveStake for, passing in the relevant token received in the response to ReserveStake for the ReserveStake that is being committed.

If any of the calls fail or return false, the game must immediately log the fact before exiting gracefully. Do not end the game cycle or display any message to the player.

5.6 RollbackStake

This is called in the event that a staking action cannot be committed. iKernel will automatically roll back any uncommitted stakes when the game exits but in the case of the player electing to exit the game when they reach a limit the game is required to collect before it exits. If the staking action is not rolled back first, the uncommitted stake will remain on the cabinet when the game exits.

5.7 SetCheckpoint

This call is used to provide game phase information in order to comply with the 2018 decree. Contrary to previous information indicated in this document, the randomness used is to be passed into this call as part of the PhaseInfo struct. It is called as each phase of gameplay is completed where that phase has drawn a random number from the central system. A series of Free Spins or Extra Spins will need to call SetCheckpoint after each spin, as each spin should draw additional randomness

The call takes as parameters a char* PhaseSummary which contains a string similar to that used in EscrowWinnings but describing the phase that has just completed, and a PhaseInfo struct. Note that for phases that are non-spin features, the PhaseSummary string will need to be modified accordingly.

PhaseInfo has 5 components. The first is an enum that labels the phase, one of either:

GAME_PHASE_PRIMARY – the games initial phase the player sees upon committing their bet. This could be the initial reel spin, the roulette wheel, a hand of cards, the first throw of dice etc.

GAME_PHASE_ANCILLARY - Used for phases that require additional stake to be taken from the players credit, such as a Split / Double-down in Blackjack, or an extra ball in Latin Bingo

GAME_PHASE_GAMBLE - Any kind of Risk state, where the player risks winnings awarded from a previous phase. This includes Special Time / Energia type functions where the player elects to pay their winnings to a separate internal bank to draw the stake for additional spins from – in effect the player is choosing to gamble their winnings on additional spins. These would be Gamble spins with any features won during them listed as FEATURE.

GAME_PHASE_FEATURE - A sub-feature of the game, which has been awarded in addition to or in place of a cash prize. This includes Free Spins and Extra Spins provided that additional stake is not taken to play as well as any kind of trail, pick-me or other gameplay device that is not the base spin. This description is also used for Fortune Spins, where players can choose to stake on multiple spins – each spin would be checkpointed as a FEATURE and there would be no PRIMARY for that gamecycle.

The second component is the prize in minor units. This can be 0 if no prize was awarded for an individual Free Spin / Extra Spin, a Risk is lost or the base game had a losing outcome

The third component is an int* to an array of the random numbers used to generate the outcome you are checkpointing. It must contain every random number used to generate the phase since either the start of the game cycle or the last call to SetCheckpoint.

The fourth component is a count of the third

The fifth component is used to indicate the stake that the player has made on this phase. For Primary or Ancillary phases this is simply the value passed to ReserveStake or ReserveAdditionalStake. For Gamble phases, the value should be the amount the player has chosen to risk.

EndGameCycle (see below) contains a parameter that allows you to pass phase information. Games may call SetCheckpoint and pass null into this parameter for their final phase, or may insert the information for their final phase into EndGameCycle – however they **must not** do both.

5.8 AwardWinnings

Awards the winnings for the game cycle to the player. The call takes the amount in minor units (cents) to credit as well as the usual userData and a no data callback.

AwardWinnings should not be called if the player enters Risk until the Risk game is completed. If the game has a state where it waits at the end of the main game (or indeed each Risk stage) to invite the player to Risk or Collect, AwardWinnings should not be called until the player collects.

AwardWinnings should only be called if there is a value of greater than 0 to award. While it may appear to make for simpler code, do not call AwardWinnings every game cycle as this will cause the game to fail Inspired testing, as it causes issues with reporting.

AwardWinnings can only be called once per game cycle. Attempts to call it more than once will (intentionally) cause the OGAPI DLLs internal state machine to break.

It is vitally important that AwardWinnings and EndGameCycle (see below) are called as closely together as possible.

If the call fails or the callback contains non-Null errorData, the game must immediately log the fact before exiting gracefully. Do not end the game cycle or display any message to the player.

5.9 EndGameCycle

Informs the iKernel that the game cycle has now finished. The call takes as a parameter the 'final' string describing the state of gameplay. This final string should also include the details of the last Risk game the player has played and the total winnings from them:
R1:QLI10|R2:ELJSS|R3:EL10J|R4:JBJLI|R5:ZBJQ|ES:0|FS:0|Total:25|[G:25,X2:W:50]

In this example, the player won 25c from the reels, took 25c to the Risk game (G for Gamble here), won X2 to double their winnings to 50c. More information on Risk can be found below in the Risk section.

EndGameCycle must not be called until all animations that are being played (such as "big win", monies moving onto the credit meter etc) have finished playing. It must be followed by a call to AskContinue, which will almost certainly result in the game calling AskContinue twice in very quick succession particularly during Autoplay, as it must be called again before the next StartGameCycle.

EndGameCycle must be called as soon as possible once AwardWinnings has called back.

EndGameCycle can take PhaseInfo, negating the need to make a separate call to SetCheckpoint. If you elect to call SetCheckpoint instead, PhaseInfo should be null. Do not pass through the same PhaseInfo if you have already called SetCheckpoint.

If the call fails or the callback returns non-Null errorData, the game must log the fact and terminate gracefully. No message should be shown to the player.

5.10 Cashout

Called when the player hits the hardware or software "Ticket Out" button or chooses to stop play when a player limit is breached. This button should not be active unless the game is idle, and not while the game is being autoplayed. Note that where games have an interim state at the end of a game cycle inviting the player to Collect winnings or enter Risk, this button must function to collect the winnings and end the game cycle rather than print the ticket. In this instance, the software button must be changed to read "Raccogli" instead of "Ticket Out", indicating its true function. It is only possible to Ticket Out outside of a game cycle.
When a valid press is detected, the game must call `LegionItalyInterface -> Cashout ( CallbackWithNoData cb, void *userData )` A successful callback may take some time to arrive. The collect has been successful if the callback contains Null error data. The iKernel may indicate to the game to exit either by OnExit in which case the game should exit immediately, or by onExitDeferred in which case the game should exit once it receives the callback to Cashout.

If errorData is not Null, this indicates that for whatever reason, iKernel has not been able to process the cashout request. If this happens the game should display a message to the player that it has been unable to print a ticket for [amount] for 5 seconds, then exit to the main menu. Note that if a request to exit by OnExitDeferred is received while the message is on screen, it should not be processed until the message has been displayed for the full 5 seconds. If OnExit is received, the game must exit immediately even if the message has not been displayed for 5 seconds. Note that this latter requirement can sometimes cause Inspired testing to falsely indicate an issue is present.

5.11 Extra Spins, Free Spins and other features

Game designers are not restricted in the features they can offer to the player if the amount paid to the player can be determined as soon as the random number(s) for the phase of the game cycle are received. Since the introduction of the recovery system it is no longer required that an entire game cycle outcome is predetermined.

Care needs to be taken if you choose to include Extra Spins. Extra Spins are additional spins won from reel spins in place of a monetary award and must not be confused with Free Spins. These must also deliver the same award if supplied the same random details. The game cycle is not ended and monies awarded until all Extra Spins have been played off, including any additional feature wins or retriggered Extra Spins won while in Extra Spins. Energia / Special Time and other similar stake-with-your-winnings features are considered part of the same game cycle

5.12 Quick Spin

If Quick Spin is enabled (see Loading and Initialization above) the game needs to allow the player to cancel animation as soon as it knows the outcome of the game. You will have started a 'fake' animation towards the start of the game cycle (see ReserveStake) and started your game moving towards the eventual outcome as soon as you have drawn sufficient randomness.

As soon as the 'fake' animation starts, the Start button must change to read Stop. It can either become active at once and inactive once pressed until the result is known, or not become or appear active until the outcome is known. Once a valid press has occurred and the outcome is known, the game should skip all animations (or play through them as fast as possible) to display the outcome.

The hardware Start button should perform the same function as the software button, and should only be lit and active at the times the software button is regardless of whether the software button reads Start or Stop.

Note that using Quick Spin to stop the reels should not skip any winnings animations that are included. The game designer may choose to have an additional press of the button skip animations but this is not required. Using Quick Spin or skipping winning animations must not automatically start a new game cycle unless Autoplay is in use.

More information on the behaviour of Quick Spin, button functions and Autoplay is available in the Content Specification document.

5.13 OnExitDeferred and OnExit

At any point while running, amongst other events the game may receive a callback to either of the callbacks registered for the exit events during initialisation.

A callback to OnExitDeferred indicates to the game that it must exit at the end of the current game cycle, once all winnings have been awarded or lost and the EndGameCycle / AskContinue calls have been made. If the game is not inside a game cycle, it should exit immediately.

OnExit should be treated as a critical failure and logging and exit should occur immediately irrespective of what stage, state or phase the game is in.

All forms of exit from the game should be 'clean'. See "Disconnection and shutdown" below.

6. Risk

Risk describes the state where the player may elect to risk their winnings for a chance to increase or lose them. It is treated as a separate game within the main game cycle, and requires its own random number and escrow. The new decree requires that Risk is established as a separate gameplay phase. Taking the basic game cycle described above, the Risk state is available after the call to CommitReserveStake and before the call to AwardWinnings.

A game must not force a player to Risk without first giving the player the opportunity to collect their winnings. The Risk feature must have the opportunity to collect at any point while not actually within a Risk game, even if the player has not yet played a Risk (i.e. the player must be able to enter Risk and decide to leave without taking a Risk). It is permissible however to automatically enter the Risk game if the above is upheld. More information on Auto Risk is available in the content specification document.

There is nothing extra the game needs to do to inform the iKernel that it is entering or exiting Risk until the player starts a Risk game (although it must continue to heartbeat). Once the player elects to take a Risk, the game should start any relevant animations immediately (in the case of something like a card-flip this may not be possible) before requesting a centrally determined random outcome using `GetRandomInt32` or `GetRandomInts32` (see above) and awaiting the response.

When the response is received, the game should first call `LegionItalyInterface -> SetCheckpoint` and pass the value that will be won from the risk as well as a description of the state of the risk. If the risk results in a losing outcome, the value in amount must be 0.

The status string must include the details of the base game as well as the risk undertaken. R1:QLI10|R2:ELJSS|R3:EL10J|R4:JBJLI|R5:ZBJQ|ES:0|FS:0|Total:25|[G:25,X2:W:50] denotes a win of 25c, taken into a 2X gamble with a winning outcome, for a total win of 50c. Subsequent Risks should merely update these values:
R1:QLI10|R2:ELJSS|R3:EL10J|R4:JBJLI|R5:ZBJQ|ES:0|FS:0|Total:25|[G:50,X2:W:100]
R1:QLI10|R2:ELJSS|R3:EL10J|R4:JBJLI|R5:ZBJQ|ES:0|FS:0|Total:25|[G:100,X4:W:400]

Each Risk, regardless of the amount risked or multiplier applied to it requires a new random number and call to SetCheckpoint. No option must be available in the Risk that will enable the player to win more than the maximum win. If that leaves no option to Risk (such as winning €3,000 from taking a €1,500 win into a 50/50 Risk) the amount must be automatically collected from the Risk and added to the player's credit.

When the player presses either the hardware Collect button or the software Incassare button the Risk stage is complete and must not be offered again for that game cycle. There is no facility to part-collect in the Italian market. The game should proceed to call `LegionItalyInterface -> AwardWinnings` and `LegionItalyInterface -> EndGameCycle` to complete the game cycle. Note that `AwardWinnings` must not be called if the Risk resulted in a loss. If the risk game was entered but not played, it is necessary to pass the phase information either by a call to SetCheckpoint or as part of EndGameCycle in order to indicate that during the risk 0 monies were staked and 0 monies were won, but that the game still completed a Risk phase. This applies whether the Risk was entered automatically, or the player chose to enter but left.

7. Tax

Currently, the iKernel handles the taxing of winnings over a certain value. Should it detect you are awarding credit over the threshold it will send a callback to OnExitDeferred. If your game is reacting correctly to this event (see Game Cycles above), there is nothing more it needs to do; the menu will display the tax popup to the player once the game finishes exiting.

However, OGAPI 2.20.0.0 and later expose an event to the game to allow a neater method of displaying a tax popup within the game, rather than having the menu display it. This prevents the player having to re-launch the game after every taxable win. At the time of writing, there are sub-estates both with and without this functionality. It is necessary for games to cater for both eventualities.

7.1 Implementation

In order to use the updated functionality, you will need to use OGAPI v2.20.0.0 or later and implement OGAPI_LegionItComma6_Interface_v2 or later.

The flag dictating whether you should or should not implement tax popups in game is received as part of your callback from GetProperties. It is named LegacyTaxPopup and its value will be TRUE or FALSE (treat the values case insensitive for comparison – TRUE, True and true are all the same). If the value is false game should display its tax popup. However if the value is true or absent, then the game should not attempt to display its own popup.

Games released to Inspired should be tested under both scenarios by the developer. **PLEASE NOTE** that the game will continue to receive the Tax event even if the flag above is true – the flag instructs the game to ignore it.

The newly created event is delivered to the game by way of a callback. This callback is registered by calling OnTaxEvent and passing a CallbackWithStringInfo. When invoked, the string value will contain XML:

```
<?xml version="1.0" encoding="UTF-8"?>
<winTax>
<tax minThresh="20000" maxThresh="500000" pct="20" fullAmount="32000"
taxableAmount="12000" taxAmount="2400"/>
</winTax>
```

The values are as follows:

minThresh : Minimum gross win eligible to be taxed at this threshold. At the time of writing, this is €200 but as all values are in minor units the value would be 20000
maxThresh : Maximum gross win eligible to be taxed at this threshold. There is currently no scenario where this would be any less than the max win of €5000, expressed as 500000 but it may be higher on systems that support system-managed progressive jackpots
pct : Percentage of the win amount falling within the threshold which will be taken as tax. This is 20% at the time of writing
fullAmount : Gross win. This is the value you will have passed in your AwardWinnings call - €320 (32000) in the above example
taxableAmount : Value of the win falling within the threshold. In the above example this is €120

(12000)
taxAmount : The cash value that has been deducted from the gross win

You will notice that this event does not contain the net win. The game will receive a callback to its registered OnCreditChange callback and be expected to call GetCredit. The value received here will contain the net win.

The values in the XML will allow you to form the tax popup. The below is an example of one that is presented by the menu, showing all the required information. Games presenting their own tax popup should follow a similar format and must display the same information, but may theme the panel to the game as long as the information remains readable.

The popup must not be automatically dismissed – the player will need to acknowledge it before it can be cleared and the game can continue to play. The hardware buttons must be disabled while this popup is on screen – pressing Start must not clear the popup as a player attempting to spin swiftly could easily dismiss it without having enough opportunity to read the information.

The game must continue to send heartbeats and call ProcessCallbacks while the popup is being displayed.

The popup is to be displayed after the player has seen any win celebration or related animation, but before the taxed value reaches the main credit meter. The tax event and credit change event will arrive shortly after the game calls AwardWinnings. This may require developers to adjust at which points the calls are made in relation to their animations - care must be taken to ensure that the player does not see their credit meter decrease. For games with Risk, you will of course not be calling AwardWinnings until the player has left the Risk stage.

The tax event is a fire-and-forget event from the platform – it has no way to know whether or not the popup is on screen. As such, its perfectly possible that the player may add credit, which must of course immediately be shown on the credit meter. To get around this, the game should reduce the value of the main bank it is displaying to the player by calculating the net win (fullAmount minus taxAmount) from the tax event. This way, should the player insert a note and the game updates the credit meter, it is still shown less the value of the taxed win.

7.2 Changes to the Development Kit

In order to set up your development kit to allow you to implement this functionality, you will need to change several parts of the configuration file.

7.2.1 Current behaviour (platform displays the tax popup)

PLEASE NOTE that the tax event will be sent to the game even if this is set. The LegacyTaxPopup property serves to tell the game not to action the event, it does NOT prevent the game receiving the event.

1. Open c:\ikernel\dist\conf\lcunitgroupconf.properties using your preferred text editor.
2. Add or edit the line:
   `content.*.LegacyTaxPopup=TRUE`
3. If necessary, edit the following line to read:
   `com.ingg.ikernel.level3.plugin.TaxConfig.taxedWinMsgEnabled=TRUE`
4. If necessary, edit the following line to read:
   `com.ingg.ikernel.level3.plugin.TaxConfig.ranges=20000-500000@20`
5. Restart the iKernel

7.2.2 New behaviour (game displays the tax popup)

1. Open c:\ikernel\dist\conf\lcunitgroupconf.properties using your preferred text editor.
2. Add or edit the line:
   `content.*.LegacyTaxPopup=FALSE`
3. If necessary, edit the following line to read:
   `com.ingg.ikernel.level3.plugin.TaxConfig.taxedWinMsgEnabled=FALSE`
4. If necessary, edit the following line to read:
   `com.ingg.ikernel.level3.plugin.TaxConfig.ranges=20000-500000@20`
5. Restart the iKernel

8. Player Limits

The Italian market requires players to be able to impose limits on their gaming. These are set in terms of time, or credit played. Inspired provide you with the graphical assets required to build the dialog boxes players use to input these values. Once they do so and press "OK" the game needs to call:

```
LegionItalyInterface -> SetLimits (CallbackWithNoData cb, void *userData,
int spendLimit, int timeLimit)
```

Passing in the values the player has chosen. If the call fails or the callback returns non-Null errorData, the game should log the fact and exit gracefully. No message should be shown to the player.

Once set, the game will receive callbacks to the callback registered using OnLimitsEvent. The LimitsInfo struct contains 4 int values:

spendLimit,
spendLimitRemaining,
timeLimit,
timeLimitRemaining

When no limits are set, the two Remaining fields are set to int-max (2147483647) and the two Limit fields are set to 0. When limits are set, one or both User Limit fields will update to the value(s) passed in `SetLimits` and the Remaining field will carry the number of units of that limit remaining.

When a callback to OnLimitsEvent is received where a Limit field is greater than 0 and its corresponding Remaining field is 0, the game needs to display to the player a dialog informing them they have breached their limit and asking whether they wish to continue, or stop and cash out (graphics assets are provided by Inspired for this dialog).

If the player chooses to Stop and Cash Out, the game needs to call Collect (see "Collect" above), write the reasoning to the log then proceed to exit as normal. Should the player choose to Continue, the game needs to call `SetLimits` again passing in 0 as both values. The player is then free to set their limits once again.

The game will need to have protection in place to ensure that it does not proceed to call CommitStake when there is a possibility of the initial ReserveStake breaching the spend limit. A simple solution to this is for the game to block execution after ReserveStake is called until the OnLimitsEvent is received as well as the callback to ReserveStake. If the limit is breached and the player chooses to stop and cash out, the game will first need to call RollbackStake to return the initial credit to the player before calling Collect. Games may elect to track limits internally and not even call ReserveStake if it will breach the limit. Once the player has elected to cash out, the iKernel may ask the game to exit

Where a time limit is breached inside a game cycle, the dialog must be displayed as soon as the game cycle is concluded (i.e. after the game has called and received callbacks for EndGameCycle and AskContinue). If received outside of a game cycle it must be displayed immediately.

Players are free to modify their limits at any time, but the game is not required to update the values to take account of any play that has taken place. Instead when the player re-opens the Set Limits interface the original limits they set should be presented to the player. Once opened, the Set Limits

box can only be closed by setting a limit or clearing all limits. If the Anulla button is pressed the game should call SetLimits passing 0 as both values to clear any limits set. It is acceptable to make this call even if the limits are already set to 0.

9. Recovery

The latest versions of the iKernel platform support game recovery, albeit in something of a basic form. If an interrupted game cycle is detected, the iKernel will relaunch the game when it is able to. However it does not do anything different beyond this, instead relying on the game. Games are permitted a limited number of attempts to recover before the iKernel suspends the VLT and awards the most recent value stored by the SetEscrow call. It will also clear the games memento data to ensure that subsequent restarts of the game do not attempt further recovery.

Games are required to store sufficient information to return to the point in the game cycle that they were interrupted in a file named "game.memento", which must be stored in the root of their working folder. Therefore, the games startup process must be that as soon as possible following receipt of its working folder location it examines game.memento to see if there is a game cycle unfinished. If there is not, or if the file is absent (such as would be the case for the first time the game has been run or following a catastrophic failure in recovery) there is no game cycle to recover and the game should launch as normal.

The game must save the game cycle ID received in the callback to StartGameCycle into its memento file, as well as its current state. The game cycle ID is passed back to the iKernel during recovery to enable the platform to correctly track the cycle.

The game can elect to save its state in any manner it chooses - the format of game.memento and its contents are not mandated, only the filename and location. However, it must not be human-readable and protection should be in place to prevent it being edited. When writing to the file a safe file writing methodology must be used.

If upon examination of game.memento the game determines that it has an unfinished cycle, it must complete this cycle before allowing normal gameplay. Once initialised, the game needs to call:

```
LegionItalyInterface -> AskContinue( CallbackWithIntInfo cb, void *userData
)

LegionItalyInterface -> StartGameCycle ( CallbackWithStringInfo cb, void
*userData, const char *gameCycleId )
```

as it normally would if the player had pressed Start. However, the char* `gameCycleID` will be populated by the game cycle ID that the game has loaded from its memento file.

The contents of game.memento should also be cleared to ensure the game does not mistakenly believe its self to be within an unfinished game cycle, and also to keep the file size down. There is no reason to retain completed game cycles within this file – the system does not support a 'replay' function.

The following is a list of scenarios under which the game may conceivably terminate and shows the remaining calls to make in order to complete the game cycle. Careful attention should be given to terminations around ReserveStake / CommitStake as interrupting credit handling invalidates both ReserveStake and CommitStake, requiring that both be called anew. Calls to SetEscrow are not shown as duplicating this call will not have an adverse effect. Calls to SetCheckpoint should not be duplicated – if a phase that has been checkpointed already is checkpointed again this will show up as a duplicate. Subsequent calls to GetRandomInt32 are also not shown – the game will need to save any responses it receives to GetRandomInt32 to its memento file and only request as much randomness as it still requires to continue.

Terminated after StartGameCycle

- StartGameCycle
- **GAME TERMINATED AND RELOADED**
- StartGameCycle(PREVIOUS GCID)
- ReserveStake
- GetRandomInt32
- CommitStake
- AwardWinnings
- EndGameCycle

Terminated after ReserveStake

- StartGameCycle
- ReserveStake
- **GAME TERMINATED AND RELOADED**
- StartGameCycle(PREVIOUS GCID)
- GetRandomInt32
- CommitStake
- AwardWinnings
- EndGameCycle

Terminated after GetRandomInt32

- StartGameCycle
- ReserveStake
- GetRandomInt32
- **GAME TERMINATED AND RELOADED**
- StartGameCycle(PREVIOUS GCID)
- ReserveStake
- CommitStake
- AwardWinnings
- EndGameCycle

Terminated after CommitStake

- StartGameCycle
- ReserveStake
- GetRandomInt32
- CommitStake
- **GAME TERMINATED AND RELOADED**
- StartGameCycle(PREVIOUS GCID)
- AwardWinnings
- EndGameCycle

Terminated after AwardWinnings

- StartGameCycle

- ReserveStake
- GetRandomInt32
- CommitStake
- AwardWinnings
- **GAME TERMINATED AND RELOADED**
- StartGameCycle(PREVIOUS GCID)
- EndGameCycle

A termination after EndGameCycle does not require any special behaviour, as the iKernel will not relaunch the game. Such a termination however would be considered a bug by Inspired testing and will result in the game being returned for resolution.

10. Logging

Your game must write log files of all betting activity to the working directory acquired during initialization. You may also write any other logging files that may be required that describe the internal workings of your game. No text file written should ever contain any random number received with the exception of game.memento which must be obfuscated per the above description. Files must not be named, nor must they contain text such as Debug, Test, Diagnostic or anything else that implies that the game is anything other than a full, complete live game with no issues.

Games are required to prune their own log files. It is suggested that logs older than 60 days are removed. If logs for the current day already exist, the game should append to that log rather than create a new one for every execution.

Unlike other territories, the log files written out for Italian games are not viewed by shop staff. They can however be retrieved by Inspired and other authorised parties for issue diagnoses and dispute resolution. For that reason, the failure of calls should always be logged in some way.

The betting activity log must be named "itboxlogYYYYMMDDbetevent.log" where YYYYMMDD is the date. It should contain details of the games loading, staking, stake rollbacks, reel positions including for each individual extra spin and free spin, winnings and any progressive jackpot win as well as all button presses made by the player. When logging free spins and extra spins, it should be made very clear what type of spin it is to differentiate it from the main game spin, what number that spin is and how many remain. In the event of retriggers where additional free or extra spins are won, this should also be made clear and the total remaining spins updated for subsequent logs.

Each line of the log should be date/time stamped down to milliseconds. The receipt of events should also be logged, as well as failure responses (it isn't necessary to log successes but it is not prohibited). When the game exits, the reason should be made clear (due to failure of call xx, because player pressed menu, player collected winnings, player stopped play after reaching their limit, received Kill message etc etc).

A sample BetEvent log, where Free Spins was won during Extra Spins is below. Note that Extra Spins is notated as FreeGames:

```
01/05/2014 [11:37:47:122]: ***** Entering Game *****
01/05/2014 [11:37:47:122]: name: ItComma6b_JungleBucksFreespins
01/05/2014 [11:37:47:122]: version: 1.17.35218.35195
01/05/2014 [11:37:47:122]: timestamp: 2013/05/16 17:11:53
01/05/2014 [11:37:47:200]: Start of game credit: €634,70
01/05/2014 [11:37:47:216]: Stake has been changed to €1
01/05/2014 [11:37:52:075]: Button pressed: IncreaseStakeButton
01/05/2014 [11:37:52:075]: Stake has been changed to €2
01/05/2014 [11:37:52:294]: Button pressed: IncreaseStakeButton
01/05/2014 [11:37:52:294]: Stake has been changed to €4
01/05/2014 [11:37:52:466]: Button pressed: IncreaseStakeButton
01/05/2014 [11:37:52:466]: Stake has been changed to €5
01/05/2014 [11:37:54:810]: Button pressed: StartButton
01/05/2014 [11:37:54:841]: Game (cycle) started
01/05/2014 [11:37:54:841]: Game Stake: €5
01/05/2014 [11:37:54:841]: About to attempt to deduct stake/cost of play €5,00 from credit
01/05/2014 [11:37:54:857]: System credit changed - decreased by €5, prev=€634,70, new=€629,70
01/05/2014 [11:37:55:028]: Button pressed: StopSpinButton
01/05/2014 [11:37:55:263]: Deducted stake/cost of play €5,00 from credit
01/05/2014 [11:37:55:560]: Base game spin in progress
01/05/2014 [11:37:55:560]: --------------------------------
01/05/2014 [11:37:55:560]: Queen      King      Ten      Lion      Ace
```

```
01/05/2014 [11:37:55:560]: Ace          SlipperySpins  Queen        Queen        Queen
01/05/2014 [11:37:55:560]: Queen        Ace          Jack         Ace          Zebra
01/05/2014 [11:37:55:560]: ---
01/05/2014 [11:37:55:560]: Winline              Symbol              Symbol count          Prize
01/05/2014 [11:37:55:560]: ---                  ---                 ---                   ---
01/05/2014 [11:37:55:560]: 10                   Queen               4                     'FreeGames' '1'
01/05/2014 [11:37:55:560]: 11                   Queen               4                     'FreeGames' '1'
01/05/2014 [11:37:55:560]: ---------------------------------
01/05/2014 [11:37:55:732]: Button pressed: StopSpinButton
01/05/2014 [11:38:00:966]: Button pressed: FreePlayButton
01/05/2014 [11:38:00:982]: Extra spins in progress [1/2]
01/05/2014 [11:38:00:982]: ---------------------------------
01/05/2014 [11:38:00:982]: SlipperySpins  JungleBucks    Ace          Zebra        Elephant
01/05/2014 [11:38:00:982]: King         Ace          Jack         Ten          Ace
01/05/2014 [11:38:00:982]: Queen        King         Ten          Zebra        Jack
01/05/2014 [11:38:00:982]: ---
01/05/2014 [11:38:00:982]: Winline              Symbol              Symbol count          Prize
01/05/2014 [11:38:00:982]: ---                  ---                 ---                   ---
01/05/2014 [11:38:00:982]: 16                   Ace                 3                     €2,50
01/05/2014 [11:38:00:982]: ---------------------------------
01/05/2014 [11:38:01:888]: Button pressed: StopSpinButton
01/05/2014 [11:38:03:591]: Button pressed: FreePlayButton
01/05/2014 [11:38:03:607]: Player has won [6] free spins and a [X4] multiplier in JungleRun feature
01/05/2014 [11:38:03:607]: Extra spins in progress [2/2]
01/05/2014 [11:38:03:607]: ---------------------------------
01/05/2014 [11:38:03:607]: Queen        King         King         Ace          Lion
01/05/2014 [11:38:03:607]: Queen        Zebra        SlipperySpins  JungleBucks    Queen
01/05/2014 [11:38:03:607]: Jack         Lion         King         SlipperySpins  SlipperySpins
01/05/2014 [11:38:03:607]: ---
01/05/2014 [11:38:03:607]: Winline              Symbol              Symbol count          Prize
01/05/2014 [11:38:03:607]: ---                  ---                 ---                   ---
01/05/2014 [11:38:03:607]: 22                   SlipperySpinsScatter    3                 'FreeSpins'

01/05/2014 [11:38:03:607]: ---------------------------------
01/05/2014 [11:38:04:060]: Button pressed: StopSpinButton
01/05/2014 [11:38:34:810]: Button pressed: NextSpin
01/05/2014 [11:38:34:857]: FreeSpins in progress [1/6]
01/05/2014 [11:38:34:857]: ---------------------------------
01/05/2014 [11:38:34:857]: Ten          King         Queen        Lion         SlipperySpins
01/05/2014 [11:38:34:857]: Ace          Zebra        King         JungleBucks    Zebra
01/05/2014 [11:38:34:857]: Zebra        Jack         Elephant     King         Queen
01/05/2014 [11:38:34:857]: ---------------------------------
01/05/2014 [11:38:35:278]: Button pressed: SkipAnimations
01/05/2014 [11:38:35:716]: FreeSpins in progress [2/6]
01/05/2014 [11:38:35:716]: ---------------------------------
01/05/2014 [11:38:35:716]: King         Queen        Elephant     Queen        SlipperySpins
01/05/2014 [11:38:35:716]: Lion         Lion         Zebra        Ace          Zebra
01/05/2014 [11:38:35:716]: Ace          JungleBucks    Ace          Lion         Queen
01/05/2014 [11:38:35:716]: ---------------------------------
01/05/2014 [11:38:36:528]: Button pressed: SkipAnimations
01/05/2014 [11:38:36:950]: FreeSpins in progress [3/6]
01/05/2014 [11:38:36:950]: ---------------------------------
01/05/2014 [11:38:36:950]: Elephant     Elephant     Ten          Jack         Elephant
01/05/2014 [11:38:36:950]: Ace          Ace          Queen        Ten          SlipperySpins
01/05/2014 [11:38:36:950]: King         JungleBucks    Lion         Zebra        Lion
01/05/2014 [11:38:36:950]: ---------------------------------
01/05/2014 [11:38:37:419]: Button pressed: SkipAnimations
01/05/2014 [11:38:37:857]: FreeSpins in progress [4/6]
01/05/2014 [11:38:37:857]: ---------------------------------
01/05/2014 [11:38:37:857]: Elephant     King         Jack         Lion         Zebra
01/05/2014 [11:38:37:857]: Ten          SlipperySpins  Lion         Zebra        King
01/05/2014 [11:38:37:857]: Lion         Ace          King         Ace          Lion
01/05/2014 [11:38:37:857]: ---
```

```
01/05/2014 [11:38:37:857]: Winline          Symbol          Symbol count        Prize
01/05/2014 [11:38:37:857]: ---          ---          ---          ---
01/05/2014 [11:38:37:857]: 10          Lion          3          €40,00
01/05/2014 [11:38:37:857]: ---------- ----------
01/05/2014 [11:38:38:247]: Button pressed: SkipAnimations
01/05/2014 [11:38:39:278]: Button pressed: SkipAnimations
01/05/2014 [11:38:40:388]: FreeSpins in progress [5/6]
01/05/2014 [11:38:40:388]: ---------- ----------
01/05/2014 [11:38:40:388]: Ace          Zebra          King          Zebra          JungleBucks
01/05/2014 [11:38:40:388]: King          Jack          Jack          Ace          Ace
01/05/2014 [11:38:40:388]: Queen          Ten          Lion          JungleBucks    Ten
01/05/2014 [11:38:40:388]: ---------- ----------
01/05/2014 [11:38:40:888]: Button pressed: SkipAnimations
01/05/2014 [11:38:41:325]: FreeSpins in progress [6/6]
01/05/2014 [11:38:41:325]: ---------- ----------
01/05/2014 [11:38:41:325]: Ace          SlipperySpins  Lion          Lion          Ten
01/05/2014 [11:38:41:325]: Zebra          Ace          Queen          Ace          Queen
01/05/2014 [11:38:41:325]: Ten          Ten          Jack          Ten          Elephant
01/05/2014 [11:38:41:325]: ---------- ----------
01/05/2014 [11:38:41:794]: Button pressed: SkipAnimations
01/05/2014 [11:38:44:310]: System credit changed - increased by €42,50, prev=€629,70, new=€672,20
01/05/2014 [11:38:44:669]: Game (cycle) ended
01/05/2014 [11:38:44:669]: Total winnings: €42,50
01/05/2014 [11:38:44:669]: End of game credit: €672,20
01/05/2014 [11:38:44:669]: ******************************
01/05/2014 [11:38:44:669]: Start of game credit: €672,20
01/05/2014 [11:38:45:872]: Button pressed: MenuButton
01/05/2014 [11:38:45:888]: ***** Leaving Game *****
```

11. Disconnecting and Shutdown

When the game is terminated for any reason, including in response to a request by the iKernel its self it must first inform the iKernel that it intends to exit, then run any unloading and garbage collection procedures before finally closing the connection to the iKernel DLL. It is important that the intention to exit is not called immediately before the call to close as this can cause the game to be falsely reported as crashing (disconnecting without warning).

First, the game must call `LegionItalyInterface -> Exiting (CallbackWithNoData cb, void *userData)`. From receipt of this call, iKernel will allow the game 5000msec to complete its exit procedure before it is forcibly terminated.

Once the game is ready to terminate and has unloaded all graphics and sound assets, the very last thing it should do before it terminates is shutdown the two interfaces which it calls ProcessCallbacks on. This closes the game's connection to the API and disconnects from the iKernel. It should call :

```
CabinetButtonsInterface-> shutdown()

adminInterface-> shutdown()
```

Once the admin interface is shutdown, no further calls can be made into iKernel. Games should continue to call the relevant processCallback methods until they have called shutdown.

The failure of any of these calls or non-Null errorData in the Exiting callback should not cause the game to stop exit processes but should be logged.